

STUBBORN

Boutique en ligne — Symfony 8

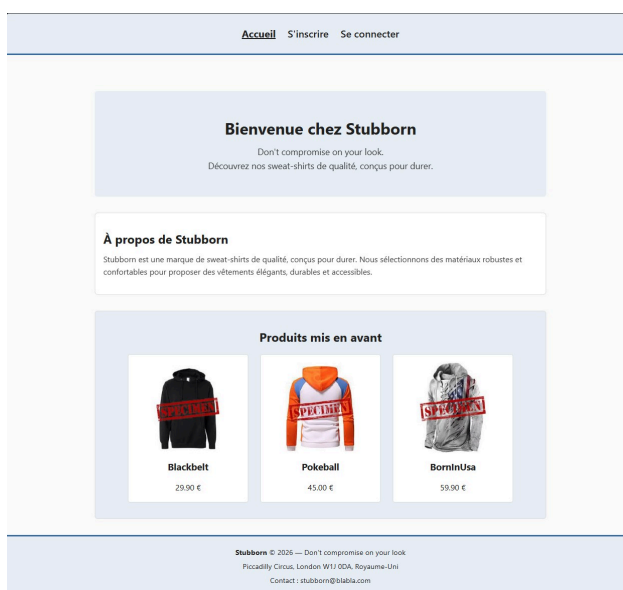
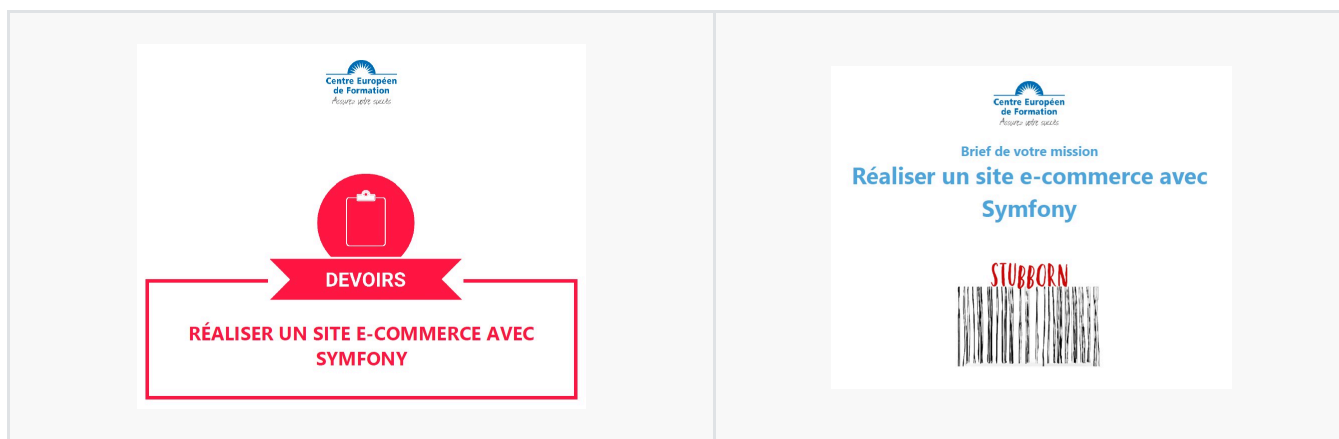
Documentation du projet

Projet réalisé dans le cadre du Centre Européen de Formation

Auteur : PERARD

Année : 2026

Version : 1.0



SOMMAIRE

STUBBORN

Boutique en ligne — Symfony 8

Documentation du projet

SOMMAIRE

1. Introduction

2. Résumé exécutif

3. Présentation générale du projet

3.1 Contexte pédagogique

3.2 Technologies utilisées

3.3 Organisation du dépôt

4. Architecture globale

4.1 Structure du projet

4.2 Architecture métier

4.2.1 Entités

4.2.2 Services

4.2.3 Sécurité

5. Plan fonctionnel des routes

5.1 Tableau synthétique des routes

5.2 Schéma ASCII des routes

5.3 Règles de sécurité par route

5.3.1 Accès des routes

5.3.1.1 Routes publiques

5.3.1.2 Routes utilisateur (ROLE_USER)

5.3.1.3 Routes utilisateur vérifié (ROLE_USER + IS_VERIFIED)

5.3.1.4 Routes administrateur (ROLE_ADMIN)

5.3.2 Définition des routes

5.4 Mapping routes → tests fonctionnels

5.4.1 Organisation des tests

5.4.1.1 Tests HOM (pages publiques)

5.4.1.2 Tests SEC (sécurité / rôles)

5.4.1.3 Tests CRT (panier)

5.4.1.4 Tests ACH (paiement Stripe)

5.4.2 Principes du codage des tests

6. Fonctionnalités

6.1 Catalogue produits

6.1.1 Fonction

6.1.2 Architecture sous-jacente

6.1.3 Principes de codage

6.2 Panier

6.2.1 Fonction

6.2.2 Architecture sous-jacente

6.2.3 Principes de codage

6.3 Authentification et gestion du compte

6.3.1 Fonction

6.3.2 Architecture sous-jacente

6.3.3 Principes de codage

6.4 Paiement Stripe

6.4.1 Fonction

6.4.2 Architecture sous-jacente

6.4.3 Principes de codage

6.5 Back-office administrateur

6.5.1 Fonction

6.5.2 Architecture sous-jacente

6.5.3 Principes de codage

6.6 Synthèse des fonctionnalités

7. Tests unitaires et fonctionnels

7.1 Architecture des tests

7.2 Groupes de tests

7.3 Détail des tests unitaires Panier et Achat

7.3.1 Tests du Panier (CRT)

7.3.2 Tests d'Achat (ACH)

7.4 Configuration Stripe en mode test

7.4.1 Configuration Stripe pour les tests automatisés

7.4.2 Configuration Stripe pour l'application

7.4.3 Synthèse des deux configurations

7.5 Scripts Composer et groupes d'environnement

7.5.1 Scripts disponibles

7.5.2 Groupes d'environnement

7.5.3 Cas d'emploi

8. Sécurité

8.1 Configuration générale

8.2 Rôles et authentification

8.3 Vérification du compte utilisateur

8.4 Voter IS_VERIFIED

8.5 Protection du back-office

8.6 Synthèse des mécanismes de sécurité

9. Expériences utilisateur

9.1 Navigation publique

9.2 Parcours client

9.3 Gestion du panier

9.4 Processus d'achat

9.5 Administration du site

9.6 Lancement de l'application et des tests

9.6.1 Lancement de l'application

9.6.2 Exécution des tests

10. Conclusion

10.1 Structuration du projet MVC

10.2 Apport de Symfony

10.3 Réemploi de l'architecture

10.4 Perspective personnelle

11. Annexes

11.1 Organisation du dépôt

11.1.1 Décomposition en milestones

11.1.2 Organisation des issues

11.1.3 Kanban de gestion du planning

11.1.4 GitGraph du développement

11.2 Scripts de `composer`

11.3 Arborescence du projet et descriptif

11.4 Extraits de codes importants

- 11.4.1 Service CartService
 - 11.4.2 Service StripeService
 - 11.4.3 Contrôleur OrderController
 - 11.4.4 TestACH-04 — Création de session Stripe simulée
 - 11.4.5 Voter IS_VERIFIED
-

1. Introduction

Le projet **Stubborn** est une boutique en ligne développée avec **Symfony 8**, dans le cadre du Centre Européen de Formation.

Il s'agit d'une application e-commerce complète, conçue pour offrir un parcours utilisateur fluide et structuré, depuis la consultation des produits jusqu'au paiement sécurisé.

Le site intègre l'ensemble des fonctionnalités attendues pour un projet professionnel :

- un catalogue produits organisé,
- un panier stocké en session,
- un système d'authentification,
- une vérification email obligatoire,
- un paiement sécurisé via Stripe (mode test),
- un back-office réservé à l'administrateur,
- une suite de tests automatisés garantissant la stabilité du projet,
- une documentation technique exhaustive.

Ce document constitue le **livrable final** du projet.

Il présente l'architecture, les routes, les fonctionnalités, les tests, la sécurité et les éléments d'expérience utilisateur, afin d'offrir une vision claire et complète de l'application.

2. Résumé exécutif

Le projet **Stubborn** a été conçu pour démontrer :

- la maîtrise de **Symfony 8**,
- la capacité à structurer un projet MVC,
- l'intégration d'un système de paiement sécurisé,
- la gestion des rôles et de la sécurité,
- la mise en place de tests automatisés,
- la production d'une documentation.

Le parcours utilisateur couvre l'ensemble du processus d'achat :

1. consultation des produits,
2. choix d'une taille,
3. ajout au panier,
4. création d'un compte,
5. vérification email,
6. paiement via **Stripe**,
7. réception d'un email de confirmation.

Le projet s'appuie sur un environnement de développement :

- développement sous **Windows + WSL**,
- utilisation de **VS Code** et **Typora** pour la documentation,
- dépôt GitHub dédié,
- intégration **Stripe** en mode test,
- serveur de messagerie local **Mailpit**.

L'ensemble constitue une base solide pour un projet pédagogique structuré, reproductible et conforme aux attentes du CEF.

3. Présentation générale du projet

3.1 Contexte pédagogique

Le CEF demande la réalisation d'un site e-commerce simple intégrant :

- la gestion des utilisateurs,
- un panier,
- un système de paiement,
- un back-office,
- une documentation.

Le projet a été organisé en **milestones** et **issues GitHub**, suivies dans un tableau **Kanban**, permettant une progression structurée et une traçabilité complète.

3.2 Technologies utilisées

- **Symfony 8**
- **PHP 8.4**
- **MySQL 8**
- **Twig**
- **Stripe Checkout**
- **Mailpit**
- **PHPUnit 13**
- **Composer**

3.3 Organisation du dépôt

Le projet **Stubborn** est hébergé sur GitHub à l'adresse suivante :

🔗 **Dépôt principal** https://github.com/MonLucCo/CEF_Symfony_Site-e-commerce_Test-version

Le suivi du projet est assuré par un **tableau Kanban GitHub Projects**, accessible à l'adresse :

🔗 <https://github.com/users/MonLucCo/projects/12/views/1>

Ce dépôt contient l'ensemble du code source, la documentation, les tests et les scripts d'exploitation. Le fichier **README.md** présente :

- le contexte du projet et ses objectifs pédagogiques ;
- les instructions d'installation et de configuration ;
- les commandes Composer principales ;
- les informations sur les tests et les environnements ;
- les liens vers les milestones et le tableau Kanban.

Pour la réalisation du projet, ce dépôt décompose les travaux en quatre (4) **milestones** et onze (11) **issues** . Le flux de travail prévoit :

- une branche principale **main** : version diffusée
- une branche de développement **dev** : version en cours d'intégration
- des branches de travaux **feature/issue-x-designation** : réalisation de la tâche élémentaire

Cette organisation GitHub — milestones, issues, Kanban et GitGraph — constitue la **colonne vertébrale du projet Stubborn**. Elle garantit une progression structurée, une visibilité complète sur le développement et une documentation parfaitement alignée avec le code source.

La structure détaillée est fournie en annexe.

4. Architecture globale

4.1 Structure du projet

La structure suivante présente **les principaux dossiers du projet Symfony**. Chaque dossier joue un rôle spécifique dans l'architecture MVC et dans le fonctionnement global de l'application.

Code

```
src/  
  Controller/  
  Entity/  
  Repository/  
  Service/  
  Security/  
templates/  
assets/  
public/  
tests/  
docs/
```

Le tableau suivant fournit la description succincte des dossiers :

Dossier	Rôle dans le projet
src/	Contient l'ensemble du code applicatif (logique métier, contrôleurs, services).
src/Controller/	Regroupe les contrôleurs qui orchestrent les actions et les flux de données entre les vues et le modèle.
src/Entity/	Définit les entités Doctrine (Product, User), représentant les données métier.
src/Repository/	Contient les classes responsables des requêtes et accès à la base de données.
src/Service/	Héberge les services métiers (CartService, StripeService, EmailVerifier).

Dossier	Rôle dans le projet
src/Security/	Regroupe les éléments de sécurité (Voter, logique d'accès).
templates/	Contient les vues Twig utilisées pour l'affichage des pages.
assets/	Fichiers front-end (CSS, JS, images) gérés par AssetMapper.
public/	Point d'entrée de l'application (index.php) et ressources publiques accessibles par le navigateur.
tests/	Ensemble des tests unitaires et fonctionnels organisés par groupes (HOM, SEC, CRT, ACH).
docs/	Documentation du projet, schémas, annexes et ressources associées.

Note

Cette section présente uniquement **les dossiers principaux** du projet. L'arborescence complète et détaillée est fournie en **annexe 11.2**.

4.2 Architecture métier

4.2.1 Entités

- **Product** : nom, description, prix, stock par taille
- **User** : email, mot de passe, rôles, isVerified

4.2.2 Services

- **CartService** : gestion du panier
- **StripeService** : création de session Stripe
- **EmailVerifier** : vérification email

4.2.3 Sécurité

- rôles : `ROLE_USER`, `ROLE_ADMIN`
- vérification email obligatoire
- voter `IS_VERIFIED`

Cette architecture modulaire permet une évolution progressive du projet tout en garantissant une séparation claire des responsabilités.

5. Plan fonctionnel des routes

Le plan fonctionnel des routes présente l'organisation complète de la navigation au sein de l'application.

Il a été construit de manière itérative au fil du développement, afin de refléter progressivement l'architecture finale, les besoins métier et les exigences de sécurité.

Cette section constitue désormais la référence centrale du projet : elle décrit l'ensemble des routes disponibles, leur arborescence, les règles d'accès qui leur sont associées et leur lien direct avec les tests fonctionnels.

L'objectif est de fournir une vision claire, structurée et exhaustive du fonctionnement interne de l'application, en montrant comment chaque route s'inscrit dans l'expérience utilisateur, dans la logique de sécurité et dans la couverture de tests.

5.1 Tableau synthétique des routes

Le tableau ci-dessous présente l'ensemble des routes de l'application **Stubborn**, classées par rôle, catégorie fonctionnelle et méthode HTTP.

Il constitue la référence principale pour :

- la compréhension de l'architecture fonctionnelle
- la sécurisation des routes)
- la rédaction des tests fonctionnels
- la documentation finale du projet

Ce tableau correspond à la version finale du plan fonctionnel des routes, intégrant :

- la suppression des données métier dans les URLs du panier
- l'ajout exhaustif des méthodes HTTP
- la classification par rôle (Public, ROLE_USER, ROLE_ADMIN, IS_VERIFIED)
- la cohérence avec les tests du Panier (CRT) et de Commande (ACH)

Le tableau suivant fournit la liste des routes de l'application :

Route	Rôle	Accès	Catégorie	Méthode
/	Accueil	Public	Public	GET
/login	Connexion	Public	Public	GET / POST
/register	Inscription	Public	Public	GET / POST
/register/resend	Renvoi Email	Public	Public	POST
/products	Liste produits	Public	Public	GET
/product/{id}	Fiche produit	Public	Public	GET
/verify/email	Activation	Public (sécurisé)	Technique	GET
/logout	Déconnexion	ROLE_USER / ROLE_ADMIN	Technique	GET (Symfony)
/account/not-verified	Compte non activé	ROLE_USER	Utilisateur	GET
/cart	Panier	ROLE_USER	Utilisateur	GET
/cart/add	Ajouter panier	ROLE_USER	Utilisateur	POST
/cart/decrease	Diminuer panier	ROLE_USER	Utilisateur	POST
/cart/remove	Retirer panier	ROLE_USER	Utilisateur	POST
/cart/clear	Vider panier	ROLE_USER	Utilisateur	POST
/order/checkout	Paie ment	ROLE_USER + IS_VERIFIED	Utilisateur vérifié	GET
/order/success	Paie ment réussi	ROLE_USER + IS_VERIFIED	Utilisateur vérifié	GET
/order/cancel	Paie ment annulé	ROLE_USER + IS_VERIFIED	Utilisateur vérifié	GET
/order/send-confirmation	Email confirmation	ROLE_USER + IS_VERIFIED	Utilisateur vérifié	POST
/admin	Back-office	ROLE_ADMIN	Administrateur	GET
/admin/product/*	CRUD produits	ROLE_ADMIN	Administrateur	GET / POST

5.2 Schéma ASCII des routes

Ce schéma ASCII offre une vue d'ensemble hiérarchique des routes, classées par rôle et par catégorie fonctionnelle. Il permet une compréhension rapide de la structure du projet.

```
/
├─ GET      /login
├─ GET/POST /register
│   └─ POST /register/resend
├─ GET      /products
├─ GET      /product/{id}
├─ GET      /verify/email
├─ GET      /account/not-verified
├─ GET      /cart
│   └─ POST /cart/add
│   └─ POST /cart/decrease
│   └─ POST /cart/remove
│   └─ POST /cart/clear
├─ GET      /order/checkout
├─ GET      /order/success
├─ GET      /order/cancel
└─ POST     /order/send-confirmation

/admin
├─ GET      /admin
└─ GET/POST /admin/product/*
```

5.3 Règles de sécurité par route

La sécurité de l'application repose sur un ensemble de règles appliquées à chaque groupe de routes, en fonction de leur rôle et de leur sensibilité. Cette section présente ces règles de manière structurée, en expliquant comment l'accès aux différentes parties du site est contrôlé selon le niveau d'authentification requis, le rôle de l'utilisateur et les contraintes spécifiques liées au paiement ou à l'administration.

L'objectif est de fournir une vision claire des mécanismes de protection mis en place :

- accès public ou restreint,
- routes nécessitant une authentification simple,
- routes réservées aux comptes vérifiés,
- routes strictement administratives,
- routes techniques protégées par des mécanismes internes (signature, session, etc.).

Cette organisation garantit un fonctionnement cohérent de l'application, tout en assurant la protection des données, la fiabilité du parcours utilisateur et la conformité des actions sensibles.

5.3.1 Accès des routes

5.3.1.1 Routes publiques

Route	Sécurité
/, /login, /register, /products, /product/{id}	Accès libre
/verify/email	Accès public mais protégé par signature HMAC
/register/resend	POST uniquement, limite les abus

5.3.1.2 Routes utilisateur (ROLE_USER)

Route	Sécurité
/cart*	Accès réservé aux utilisateurs connectés
/account/not-verified	Redirection automatique si compte vérifié
/logout	Géré par Symfony

5.3.1.3 Routes utilisateur vérifié (ROLE_USER + IS_VERIFIED)

Route	Sécurité
/order/*	Protégé par un Voter IS_VERIFIED
/order/checkout	Vérifie panier + quantités
/order/send-confirmation	POST obligatoire

5.3.1.4 Routes administrateur (ROLE_ADMIN)

Route	Sécurité
/admin*	Accès strict ROLE_ADMIN
/admin/product/*	CRUD complet, POST pour les actions mutantes

5.3.2 Définition des routes

La définition des routes repose sur une configuration centralisée dans le fichier `security.yaml`, complétée par des annotations et des contrôles applicatifs au sein des contrôleurs Symfony. Chaque route est associée à un niveau d'accès déterminé par les rôles, les conditions de vérification du compte et, le cas échéant, par un **Voter** spécifique. Cette approche garantit une cohérence entre la logique métier, la sécurité et la navigation utilisateur.

Les principes de codage sont les suivants :

1. **Déclaration des routes** Les routes sont définies dans les contrôleurs à l'aide des annotations `#[Route()]`. Chaque annotation précise le chemin, le nom de la route et la méthode HTTP autorisée.

Exemple `PHP` :

```
#[Route('/cart', name: 'app_cart_index', methods: ['GET'])]
public function index(CartService $cartService): Response
{
    // ...
}
```

2. **Configuration de la sécurité** Le fichier `security.yaml` définit les règles globales d'accès :

- hiérarchie des rôles (`ROLE_USER`, `ROLE_ADMIN`),
- zones protégées par pare-feu,
- redirections automatiques en cas d'accès non autorisé.

Exemple `yaml` :

```
access_control:
- { path: ^/admin, roles: ROLE_ADMIN }
- { path: ^/order, roles: ROLE_USER }
```

3. **Contrôles applicatifs** Les contrôleurs vérifient les conditions spécifiques à chaque route :

- présence d'un utilisateur connecté,
- vérification du statut `isverified`,
- validation du contenu du panier avant paiement.

Exemple `PHP` :

```
if (!$this->getUser()->isVerified()) {
    return $this->redirectToRoute('app_account_not_verified');
}
```

4. **Voter et logique avancée** Le **Voter `IS_VERIFIED`** complète la configuration en appliquant une logique fine sur les routes sensibles (paiement, commande). Il permet de centraliser la vérification du compte et d'éviter la duplication de conditions dans les contrôleurs.

 Note

La mise en œuvre détaillée de ces mécanismes de sécurité — configuration du fichier `security.yaml`, logique du `Voter IS_VERIFIED`, et gestion des rôles — est présentée dans la **section 8 : Sécurité**.

5.4 Mapping routes → tests fonctionnels

Cette section établit la correspondance entre les routes et les tests fonctionnels (HOM, SEC, CRT, ACH). Elle permet de démontrer la **couverture fonctionnelle** du projet.

5.4.1 Organisation des tests

5.4.1.1 Tests HOM (pages publiques)

Route	Test
<code>/</code>	HOM-01
<code>/products</code>	HOM-02
<code>/product/{id}</code>	HOM-03

5.4.1.2 Tests SEC (sécurité / rôles)

Route	Test
<code>/admin</code>	SEC-01
<code>/admin/product/*</code>	SEC-02
<code>/account/not-verified</code>	SEC-03
<code>/order/*</code>	SEC-04

5.4.1.3 Tests CRT (panier)

Route	Test
/cart	CRT-01
/cart/add	CRT-02
/cart/decrease	CRT-03
/cart/remove	CRT-04
/cart/clear	CRT-05

5.4.1.4 Tests ACH (paiement Stripe)

Route	Test
/order/checkout	ACH-01 / ACH-02 / ACH-03 / ACH-04
/order/success	ACH-05
/order/cancel	ACH-06
/order/send-confirmation	ACH-05

5.4.2 Principes du codage des tests

La mise en œuvre des tests fonctionnels repose sur une architecture commune et des conventions de codage homogènes, garantissant la cohérence entre les routes définies et les scénarios de validation. Chaque test est conçu pour reproduire le comportement réel de l'application, en simulant les requêtes HTTP et en vérifiant les réponses attendues.

Les principes généraux sont les suivants :

Élément	Description
Structure des tests	Les tests sont regroupés par domaine fonctionnel (HOM, SEC, CRT, ACH) et héritent d'une classe de base <code>webTestCaseBase</code> .
Environnement isolé	Chaque exécution s'appuie sur une base de données dédiée à l'environnement <code>test</code> , initialisée par les scripts Composer.

Élément	Description
Fixtures	Les données de test sont chargées automatiquement pour garantir la reproductibilité des scénarios.
Helpers	Des helpers spécifiques (panier, authentification, i18n) facilitent la manipulation des données et la vérification des résultats.
Assertions	Les tests vérifient le statut HTTP, le contenu des pages, les redirections et les messages flash.
Organisation du code	Les fichiers de tests suivent la structure des routes : chaque groupe de tests correspond à un ensemble de routes fonctionnelles.

Note

Les principes détaillés de codage, la structure des classes de tests et les exemples d'implémentation sont présentés dans la **section 7 — Tests unitaires et fonctionnels**, qui approfondit la logique, les groupes et les méthodes utilisées pour la validation du projet.

6. Fonctionnalités

Les fonctionnalités du projet **Stubborn** traduisent les exigences du devoir en une architecture cohérente et modulaire. Chaque domaine fonctionnel repose sur une logique métier claire, des services dédiés et une intégration fluide dans le framework Symfony. Cette section présente les principales briques applicatives, leur structure et les principes de codage qui assurent la stabilité et la maintenabilité du projet.

6.1 Catalogue produits

6.1.1 Fonction

Le catalogue constitue la base du site e-commerce. Il permet à l'utilisateur, avant l'ajout au panier :

- d'afficher la liste des produits,
- de consulter une fiche détaillée,
- de choisir une taille,
- d'ajouter un article au panier.

6.1.2 Architecture sous-jacente

- **Entité Product**: définit les attributs du produit (nom, description, prix, stock par taille).
- **Repository ProductRepository**: gère les requêtes de lecture et de filtrage.
- **Controller ProductController**: orchestre l'affichage des produits et des fiches.
- **Template Twig**: présente les données sous forme de grille responsive.

6.1.3 Principes de codage

- Utilisation du pattern **MVC** pour séparer logique métier et présentation.
- Requêtes Doctrine optimisées pour la pagination et le tri.
- Validation des données via le composant **Validator**.
- Gestion des tailles et du stock par mapping dynamique.

6.2 Panier

6.2.1 Fonction

Le panier permet à l'utilisateur de gérer ses articles avant l'achat et permet :

- d'ajouter un produit,
- de diminuer ou supprimer une quantité,
- de vider le panier,
- de visualiser le total en temps réel,
- de bénéficier d'une UX claire et responsive.

6.2.2 Architecture sous-jacente

- **Service CartService**: gère la logique métier du panier (session, calcul des totaux).
- **Controller CartController**: relie les actions utilisateur aux méthodes du service.
- **Session Symfony**: stocke les données du panier côté serveur.
- **Templates Twig**: affichent le contenu du panier et les messages flash.

6.2.3 Principes de codage

- Stockage en session pour éviter la persistance inutile.
- Méthodes atomiques: `addItem()`, `decreaseItem()`, `removeItem()`, `clearCart()`.
- Vérification des quantités et du stock avant chaque opération.
- Messages flash pour informer l'utilisateur des actions effectuées.

6.3 Authentification et gestion du compte

6.3.1 Fonction

Le système d'authentification permet :

- l'inscription,
- la connexion,
- la déconnexion,
- la vérification email obligatoire avant achat.

6.3.2 Architecture sous-jacente

- **SecurityBundle** : gère la configuration des rôles et des pare-feu.
- **Controller SecurityController** : traite les formulaires d'inscription et de connexion.
- **Service EmailVerifier** : envoie et valide les liens de confirmation.
- **Entity User** : stocke les informations du compte et le statut de vérification.

6.3.3 Principes de codage

- Utilisation du composant **Form** pour les formulaires sécurisés.
- Encodage des mots de passe via **PasswordHasherInterface**.
- Vérification email obligatoire avant tout achat.
- Redirection automatique vers `/account/not-verified` si le compte n'est pas activé.

6.4 Paiement Stripe

6.4.1 Fonction

Le paiement est géré via **Stripe Checkout** en mode test, garantissant un processus sécurisé et conforme aux standards du e-commerce :

- de création d'une session Stripe,
- de redirection vers Stripe,
- de gestion des retours success / cancel,
- d'envoi d'un email de confirmation.

6.4.2 Architecture sous-jacente

- **Service StripeService**: crée la session Stripe et gère les retours (success/cancel).
- **Controller OrderController**: orchestre le processus d'achat et la validation du panier.
- **Mailpit**: simule l'envoi des emails de confirmation.
- **Templates Twig**: affichent les pages de succès et d'annulation.

6.4.3 Principes de codage

- Intégration de l'API Stripe via le package `stripe/stripe-php`.
- Utilisation de sessions simulées pour les tests (mode mock).
- Redirections sécurisées vers les routes `/order/success` et `/order/cancel`.
- Envoi automatique d'un email de confirmation après paiement réussi.

6.5 Back-office administrateur

6.5.1 Fonction

Le back-office permet à l'administrateur de gérer le catalogue produits en assurant :

- l'ajout d'un produit,
- la modification d'un produit,
- la suppression d'un produit.

6.5.2 Architecture sous-jacente

- **Controller AdminController** : gère les opérations CRUD sur les produits.
- **Form ProductType** : définit les champs et les validations du formulaire.
- **SecurityBundle** : protège l'accès par le rôle `ROLE_ADMIN`.
- **Templates Twig** : affichent les interfaces d'administration.

6.5.3 Principes de codage

- Accès strict réservé aux administrateurs.
- Utilisation du composant **Form** pour les opérations CRUD.
- Validation des données avant persistance.
- Messages flash pour confirmer les actions.

6.6 Synthèse des fonctionnalités

Domaine	Service principal	Contrôleur associé	Objectif
Catalogue produits	<code>ProductRepository</code>	<code>ProductController</code>	Affichage et consultation des produits
Panier	<code>CartService</code>	<code>CartController</code>	Gestion des articles et des totaux
Authentification	<code>EmailVerifier</code>	<code>SecurityController</code>	Inscription, connexion, vérification
Paieement	<code>StripeService</code>	<code>OrderController</code>	Processus d'achat et confirmation
Back-office	—	<code>AdminController</code>	Gestion du catalogue et administration

 Note

Les extraits de code détaillés des services, contrôleurs et templates associés à ces fonctionnalités sont présentés en **annexe 11**, afin d'illustrer la structure et les bonnes pratiques de développement du projet Symfony.

7. Tests unitaires et fonctionnels

La section précédente a présenté la correspondance entre les routes et les tests fonctionnels, ainsi que les principes généraux de leur codage. Cette nouvelle section approfondit cette logique en détaillant la **structure technique**, la **méthodologie** et la **mise en œuvre concrète** des tests unitaires et fonctionnels du projet.

Les tests constituent un pilier essentiel de la validation du projet : ils assurent la stabilité du code, la fiabilité des fonctionnalités et la conformité du comportement de l'application avec les exigences du Centre Européen de Formation. Ils traduisent en pratique les principes exposés dans le plan fonctionnel des routes, en reproduisant les scénarios réels de navigation, de sécurité, de panier et de paiement.

Cette section présente :

- l'**architecture des tests** et les outils utilisés ;
- l'**organisation des groupes de tests** (HOM, SEC, CRT, ACH) ;
- le **détail des tests du panier et d'achat** ;
- la **configuration Stripe en mode test** ;
- les **scripts Composer** permettant leur exécution automatisée.

L'ensemble forme une démarche cohérente et professionnelle, reliant la conception fonctionnelle du projet à sa validation technique.

7.1 Architecture des tests

La suite de tests repose sur une architecture dédiée, conçue pour être réutilisable, claire et évolutive :

- `WebTestCaseBase` : classe de base centralisant les helpers, la configuration et les assertions communes.
- **Helpers de panier** : ajout, suppression, diminution, vidage, vérification des totaux.
- **Helpers d'authentification** : connexion, création d'utilisateur, vérification d'accès.
- **Helper i18n** : gestion des messages traduits dans les assertions.
- **Fixtures de test** : chargement d'un jeu de données minimal et stable pour garantir la reproductibilité.

Cette architecture permet d'écrire des tests courts, lisibles et centrés sur le comportement attendu.

7.2 Groupes de tests

Les tests sont organisés en quatre groupes fonctionnels, chacun correspondant à un périmètre spécifique du projet et à un objectif de validation distinct.

Groupe de tests	Domaine couvert	Objectif principal	Type de validation	Exigence du devoir
HOM	Navigation et pages publiques	Vérifier la configuration de PHPUnit, les helpers et la navigation de base.	Préparatoire / méthodologique	Non exigé
SEC	Sécurité, rôles et accès	Contrôler la cohérence de la sécurisation (rôles, accès, vérification email, voter).	Validation de la sécurité	Non exigé
CRT	Panier	Tester l'intégralité du fonctionnement du panier (ajout, suppression, total, cohérence).	Validation fonctionnelle	Exigé
ACH	Achat et paiement Stripe	Vérifier le processus d'achat complet et l'intégration Stripe (mode test).	Validation fonctionnelle / transactionnelle	Exigé

 Note

Les groupes **HOM** et **SEC** ont été développés pour assurer la cohérence globale du projet et la fiabilité de l'environnement de test avant la mise en œuvre des scénarios critiques. Les groupes **CRT** et **ACH**, quant à eux, répondent directement aux **exigences du devoir** du Centre Européen de Formation.

7.3 Détail des tests unitaires Panier et Achat

Cette section présente les tests automatisés couvrant les fonctionnalités critiques du projet : la gestion du panier et le processus d'achat. Chaque groupe de tests est décrit dans un tableau synthétique, puis les principes de codage sont expliqués pour assurer la cohérence et la reproductibilité des scénarios.

7.3.1 Tests du Panier (CRT)

Les tests du panier valident la logique métier liée à la gestion des articles en session. Ils garantissent la fiabilité des opérations d'ajout, de suppression, de diminution et de vidage du panier, ainsi que la cohérence des totaux affichés.

Code du test	Objectif	Route concernée	Méthode HTTP	Résultat attendu
CRT-01	Vérifier l'accès à la page du panier pour un utilisateur connecté.	/cart	GET	Affichage du contenu du panier ou message "panier vide".

Code du test	Objectif	Route concernée	Méthode HTTP	Résultat attendu
CRT-02	Tester l'ajout d'un produit au panier.	<code>/cart/add</code>	POST	Le produit est ajouté et le total est mis à jour.
CRT-03	Tester la diminution de la quantité d'un produit.	<code>/cart/decrease</code>	POST	La quantité diminue et le total est recalculé.
CRT-04	Tester la suppression d'un produit du panier.	<code>/cart/remove</code>	POST	Le produit est retiré et le total est ajusté.
CRT-05	Tester le vidage complet du panier.	<code>/cart/clear</code>	POST	Le panier est vidé et un message de confirmation est affiché.

Principes de codage :

- Les tests utilisent la classe `WebTestCaseBase` pour initialiser le client et charger les fixtures.
- Chaque test simule une requête HTTP vers la route concernée et vérifie la réponse attendue.
- Les assertions portent sur :
 - le statut HTTP (`assertResponseIsSuccessful()`),
 - le contenu du panier (`assertSelectorTextContains()`),
 - la cohérence du total (`assertEquals()` sur la valeur calculée).
- Les helpers du panier permettent de manipuler les données de session sans dépendre du front-end.
- Les messages flash sont vérifiés pour confirmer la bonne exécution des actions.

7.3.2 Tests d'Achat (ACH)

Les tests d'achat valident le processus complet de commande et de paiement via Stripe. Ils garantissent la cohérence du parcours utilisateur, la sécurité des transactions et la gestion correcte des retours de paiement.

Code du test	Objectif	Route concernée	Méthode HTTP	Résultat attendu
ACH-01	Vérifier l'accès au checkout pour un utilisateur connecté et vérifié.	<code>/order/checkout</code>	GET	Accès autorisé et création de session Stripe.
ACH-02	Tester la redirection en cas de panier vide.	<code>/order/checkout</code>	GET	Redirection vers la boutique avec message d'erreur.

Code du test	Objectif	Route concernée	Méthode HTTP	Résultat attendu
ACH-03	Tester la validation des quantités avant paiement.	/order/checkout	GET	Redirection sécurisée si incohérence détectée.
ACH-04	Vérifier la création de la session Stripe simulée.	/order/checkout	GET	Session Stripe mockée et redirection vers URL de test.
ACH-05	Tester le paiement réussi.	/order/success	GET	Panier vidé et email de confirmation envoyé.
ACH-06	Tester le paiement annulé.	/order/cancel	GET	Panier conservé et redirection vers la boutique.

Principes de codage :

- Les tests ACH utilisent un **mock Stripe** pour éviter tout appel réel à l'API.
- Les fixtures préchargent un utilisateur vérifié et un panier valide.
- Les assertions vérifient :
 - la redirection correcte (`assertResponseRedirects()`),
 - la présence du message flash,
 - la modification du panier après paiement,
 - l'envoi de l'email de confirmation (via Mailpit).
- Les tests ACH-04 à ACH-06 utilisent des sessions simulées (cf. section 7.4.1) :

```
Session::constructFrom([
    'id' => 'cs_test_fake',
    'url' => 'https://checkout.stripe.com/test-session',
]);
```

- Les helpers d'achat assurent la cohérence entre les données du panier et les réponses Stripe.
- Les tests sont exécutés dans un environnement isolé pour garantir la reproductibilité.

Note

Les exemples de codage complets des tests CRT et ACH — incluant les classes, fixtures et helpers — sont fournis en **annexe 11.3**, afin d'illustrer la structure et les bonnes pratiques de développement des tests unitaires et fonctionnels.

7.4 Configuration Stripe en mode test

L'intégration de **Stripe Checkout** repose sur une configuration en **mode test**, utilisée à la fois pour les tests automatisés et pour l'application en environnement de développement. Cette distinction garantit la sécurité des essais et la reproductibilité des scénarios sans effectuer de transactions réelles.

7.4.1 Configuration Stripe pour les tests automatisés

Les tests fonctionnels du groupe **ACH** utilisent une configuration **Stripe** simulée afin d'éviter tout appel à l'API réelle. Cette configuration permet de reproduire le comportement du paiement sans dépendre du réseau ni du tableau de bord **Stripe**.

- Les clés API sont remplacées par des valeurs factices.
- Les sessions **Stripe** sont construites à partir d'objets simulés (`Session::constructFrom`).
- Les redirections vers **Stripe** sont remplacées par une URL de test.
- Les tests vérifient uniquement la logique applicative : création de session, redirection, vidage du panier, envoi d'email.

Exemple `PHP`:

```
Session::constructFrom([
    'id' => 'cs_test_fake',
    'url' => 'https://checkout.stripe.com/test-session',
]);
```

Cette approche permet de valider le comportement du code sans effectuer de paiement réel, tout en garantissant la cohérence du parcours utilisateur.

7.4.2 Configuration Stripe pour l'application

Dans l'application réelle (mode développement ou démonstration), **Stripe** est configuré en **mode test officiel**, avec des clés API fournies par le tableau de bord **Stripe**. Ce mode permet d'effectuer de vraies requêtes vers l'API **Stripe**, mais uniquement avec des **cartes de test** et des **transactions simulées**.

- Les clés sont définies dans le fichier `.env` :

```
STRIPE_SECRET_KEY=sk_test_XXXXXXXXXXXXX
STRIPE_PUBLIC_KEY=pk_test_XXXXXXXXXXXXX
```

- Le service `StripeService` crée une session réelle via l'API **Stripe**:

```
$session = $stripe->checkout->sessions->create([
    'payment_method_types' => ['card'],
    'line_items' => $cartService->getStripeLineItems(),
    'mode' => 'payment',
    'success_url' => $this->generateUrl('app_order_success', [],
    UrlGeneratorInterface::ABSOLUTE_URL),
    'cancel_url' => $this->generateUrl('app_order_cancel', [],
    UrlGeneratorInterface::ABSOLUTE_URL),
]);
```

- Les transactions apparaissent dans le **Dashboard Stripe**, dans la section *Workbench* → *Aperçu*, sans impact financier réel.
- Les cartes de test documentées par Stripe (ex. 4242 4242 4242 4242) permettent de simuler différents scénarios de paiement.

Cette configuration est utilisée pour les démonstrations et les validations manuelles du processus d'achat.

7.4.3 Synthèse des deux configurations

Contexte	Type de clé	Session Stripe	Objectif	Impact réel
Tests automatisés	Clés factices (mock)	Objet simulé (<code>constructFrom</code>)	Vérifier la logique applicative	Aucun
Application (mode test)	Clés Stripe officielles (<code>sk_test</code> , <code>pk_test</code>)	Session réelle via API	Valider le parcours utilisateur complet	Aucun paiement réel

Note

La configuration **Stripe** en *mode test* assure une séparation stricte entre les **tests automatisés** et l'**application réelle**, garantissant la sécurité et la reproductibilité des scénarios. Les exemples de codage complets du service Stripe et des tests ACH sont fournis en **annexe 11.3**.

7.5 Scripts Composer et groupes d'environnement

Les scripts définis dans le fichier `composer.json` constituent un ensemble d'outils automatisés permettant de gérer les services, les fixtures et les tests du projet. Ils ne représentent pas un axe majeur du code applicatif, mais jouent un rôle essentiel dans la **mise en œuvre**, la **reproductibilité** et la **exploitation technique** des travaux.

7.5.1 Scripts disponibles

Les principaux scripts sont regroupés par domaine fonctionnel :

Nom du script	Commande exécutée	Objectif principal
app:fixtures	php bin/console doctrine:fixtures:load --env=dev --group=dev --no-interaction	Charger les fixtures de développement pour initialiser la base de données locale.
test:init-db	Création de la base de test (doctrine:schema:create)	Préparer un environnement de test isolé.
test:fixtures	Chargement des fixtures de test (--env=test --group=test)	Injecter les données nécessaires aux tests automatisés.
test:run	Enchaînement: init-db → fixtures → exécution des tests	Lancer la suite complète de tests.
test:run-verbose	Idem, avec affichage détaillé (--testdox)	Exécuter les tests avec titres et descriptions lisibles.
services:start	Démarrage des services (MySQL, Mailpit, Messenger, serveur Symfony)	Initialiser l'environnement applicatif complet.
services:stop	Arrêt des services actifs	Libérer les ressources locales.
services:restart	Stop → Clear cache → Dump autoload → Start	Réinitialiser proprement l'environnement.
start:test	Redémarrage des services + exécution des tests	Lancer une validation complète du projet.
start:test-verbose	Idem, avec affichage détaillé	Vérifier le projet avec suivi visuel des tests.

7.5.2 Groupes d'environnement

Les scripts sont organisés autour de deux groupes principaux :

Groupe	Environnement	Utilisation	Objectif
dev	--env=dev	Développement local et chargement des fixtures métier.	Préparer la base de données et les services pour le travail quotidien.

Groupe	Environnement	Utilisation	Objectif
test	<code>--env=test</code>	Exécution des tests automatisés dans un environnement isolé.	Garantir la reproductibilité et la non-régression du code.

Cette distinction permet de séparer clairement les données de développement (non critiques) des données de test (contrôlées et réinitialisées à chaque exécution).

7.5.3 Cas d'emploi

Les scripts Composer sont utilisés dans trois contextes principaux :

1. Initialisation du projet

- Démarrage des services via `services:start`.
- Chargement des fixtures de développement avec `app:fixtures`.
- Vérification du bon fonctionnement global avant toute modification.

2. Exécution des tests

- Préparation de la base de test avec `test:init-db`.
- Chargement des fixtures spécifiques à l'environnement de test.
- Lancement des tests via `test:run` ou `test:run-verbose`.

3. Validation complète du projet

- Utilisation du script `start:test` pour redémarrer les services et exécuter la suite de tests en une seule commande.
- Ce script est particulièrement utile pour les démonstrations ou les vérifications finales avant livraison.

Note

Bien que ces scripts ne constituent pas un axe majeur du développement applicatif, ils sont **indispensables à l'exploitation du projet**. Ils assurent la cohérence entre les environnements, la reproductibilité des tests et la fiabilité du workflow global. Les exemples de codage et d'exécution des scripts sont présentés en **annexe 11.1**.

8. Sécurité

La sécurité de l'application **Stubborn** repose sur une architecture claire et hiérarchisée, combinant la configuration du framework Symfony, la gestion des rôles, la vérification des comptes et la logique applicative. Elle garantit la protection des données, la fiabilité du parcours utilisateur et la conformité du projet avec les bonnes pratiques du développement web sécurisé.

Note

Les principes de configuration et de codage présentés ici complètent la **section 5.3.2 — Définition des routes**, où la logique d'accès et de sécurisation est appliquée à chaque route de l'application.

8.1 Configuration générale

La sécurité est principalement gérée dans le fichier `security.yaml`. Ce fichier définit :

- les **pare-feu** protégeant les zones sensibles du site ;
- les **rôles** et leur hiérarchie (`ROLE_USER`, `ROLE_ADMIN`) ;
- les **règles d'accès** associées aux chemins (`access_control`) ;
- les **mécanismes d'authentification** et de déconnexion.

Exemple simplifié `yaml` :

```
security:
  role_hierarchy:
    ROLE_ADMIN: ROLE_USER

  access_control:
    - { path: ^/admin, roles: ROLE_ADMIN }
    - { path: ^/order, roles: ROLE_USER }
```

Cette configuration établit une séparation nette entre les espaces publics, utilisateurs et administrateurs.

8.2 Rôles et authentification

L'application distingue deux rôles principaux :

- **ROLE_USER** : attribué à tout utilisateur inscrit et connecté. Il permet l'accès au panier, à la commande et aux pages de compte.
- **ROLE_ADMIN** : réservé au back-office. Il autorise la gestion complète du catalogue produits (CRUD).

L'authentification repose sur le composant **Security** de Symfony, avec un formulaire standard et une gestion de session sécurisée. La déconnexion est gérée automatiquement par le framework via la route `/logout`.

8.3 Vérification du compte utilisateur

Avant tout achat, l'utilisateur doit avoir confirmé son adresse email. Cette vérification est assurée par le service **EmailVerifier**, qui :

1. génère un lien signé (HMAC) envoyé par email ;
2. valide la signature lors du clic ;
3. active le compte et met à jour le champ `isverified`.

Ce mécanisme empêche toute utilisation frauduleuse d'un compte non confirmé et garantit la fiabilité des transactions.

8.4 Voter IS_VERIFIED

Le **VoterIS_VERIFIED** complète la configuration en appliquant une logique fine sur les routes sensibles, notamment celles liées au paiement et à la commande. Il vérifie que :

- l'utilisateur est connecté ;
- son compte est vérifié (`isverified = true`) ;
- la route concernée nécessite cette condition.

Exemple `PHP` :

```
public function voteOnAttribute(string $attribute, $subject, TokenInterface $token): bool
{
    $user = $token->getUser();
    return $user instanceof User && $user->isVerified();
}
```

Ce voter centralise la logique de vérification et évite la duplication de conditions dans les contrôleurs.

8.5 Protection du back-office

Le back-office est strictement réservé aux administrateurs. Toute tentative d'accès non autorisé entraîne :

- une **redirection** vers la page d'accueil,
- un **message flash** informant l'utilisateur,
- une **journalisation** de l'événement dans les logs Symfony.

Cette protection garantit l'intégrité du catalogue produits et la sécurité des opérations d'administration.

8.6 Synthèse des mécanismes de sécurité

Mécanisme	Objectif principal	Niveau d'application
<code>security.yaml</code>	Configuration globale des rôles et pare-feu	Framework
Authentification Symfony	Connexion / déconnexion sécurisées	Application
Vérification email	Validation du compte avant achat	Service
Voter IS_VERIFIED	Contrôle d'accès aux routes sensibles	Logique métier
ROLE_ADMIN	Protection du back-office	Application
Redirections / Flash	Gestion UX en cas d'accès refusé	Interface utilisateur

9. Expériences utilisateur

Cette section illustre les principales expériences utilisateur du projet **Stubborn**, en présentant les parcours essentiels du site sous forme de captures d'écran et de séquences visuelles. Elle permet de comprendre concrètement le fonctionnement de l'application, depuis la navigation publique jusqu'à l'administration du site, en passant par le panier et le processus d'achat.

Les captures d'écran associées sont intégrées dans la version PDF finale et reflètent fidèlement l'interface réelle du projet.

9.1 Navigation publique

Cette première expérience utilisateur concerne les pages accessibles sans authentification. Elle illustre la découverte du site par un visiteur :

- **Page d'accueil** : présentation du site et accès rapide aux sections principales.
- **Inscription / Connexion** : formulaires simples et sécurisés.

 Note

Parcours et expérience utilisateur : Accueil → Inscription → Connexion

Description de l'étape

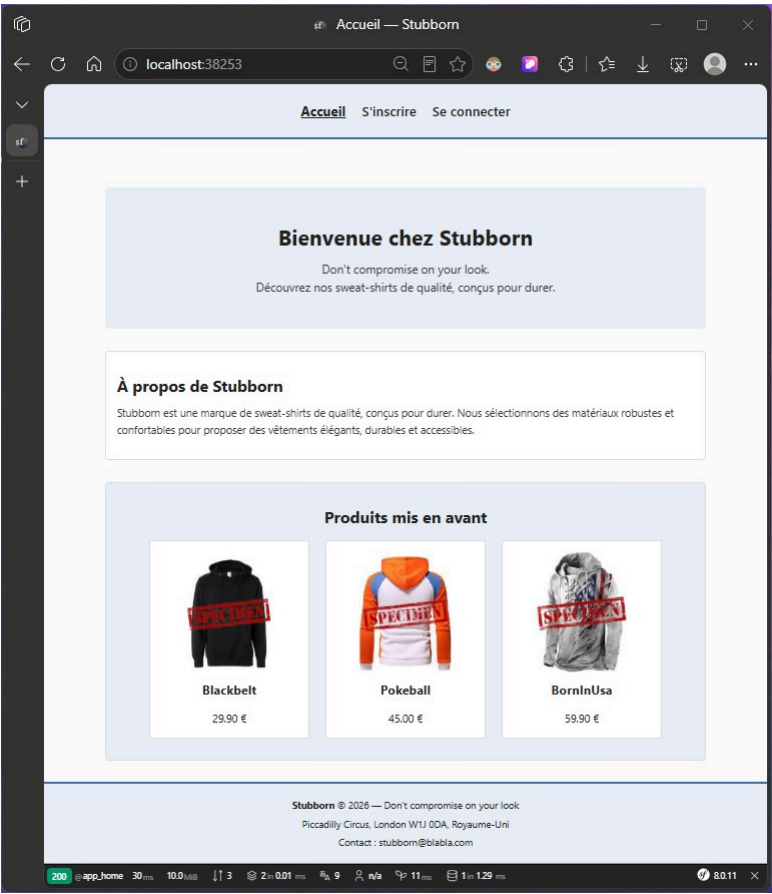
Capture d'écran

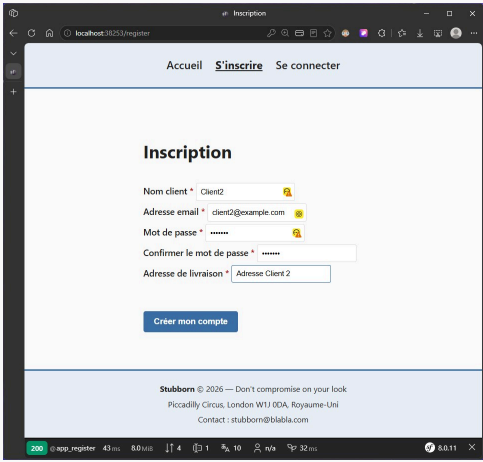
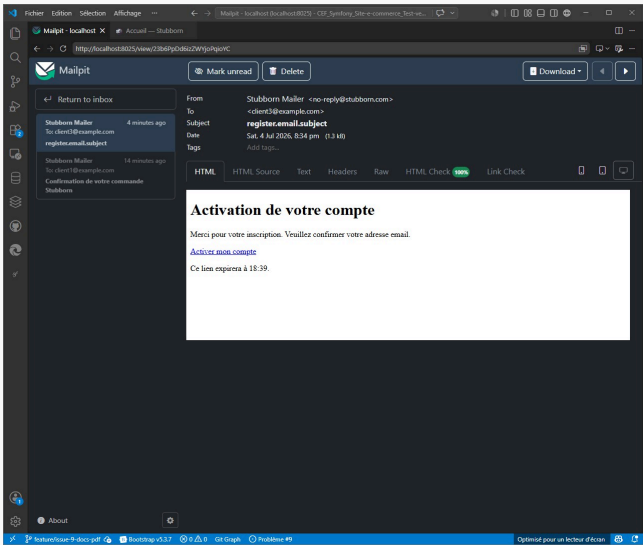
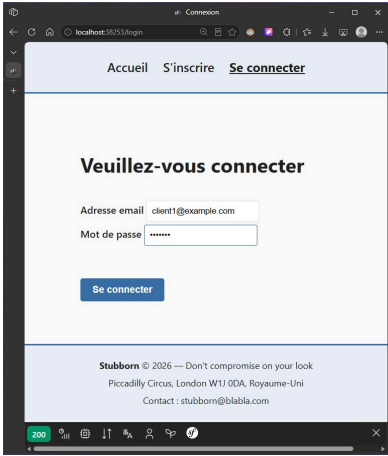
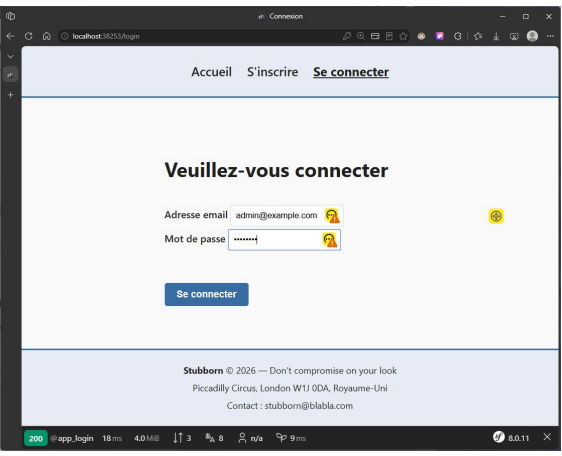
Accueil

Espace public

Présentation :

- de la société
- des produits mis en avant



Description de l'étape	Capture d'écran
Inscription Espace public Permet : - de créer un compte - de recevoir un <i>email d'inscription</i>	 
Connexion Espace public Permet : - de se connecter à son compte : * client : redirection vers <i>Accueil Client</i> * admin : redirection vers <i>Back-office</i> - d'accéder à la <i>déconnexion</i>	 

9.2 Parcours client

Cette sous-section montre le parcours complet d'un utilisateur authentifié :

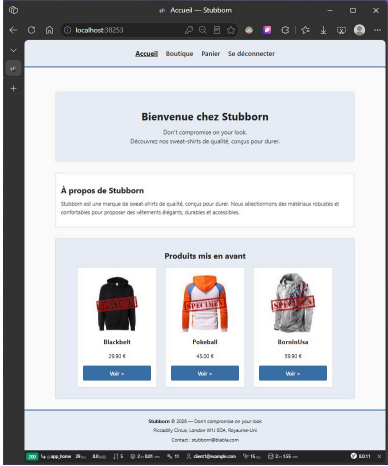
- Connexion réussie
- Accès au compte
- Navigation vers la boutique
- Ajout d'un produit au panier
- Gestion des tailles et quantités
- Accès aux pages réservées (panier, checkout)

 Note

Parcours et expérience utilisateur : Connexion → Boutique → Produit → Ajout au panier → Panier → Achat

Définitions :

- visiteur authentifié : connexion effectuée à un *compte*.
- visiteur authentifié inscrit : connexion effectuée à un *compte vérifié*.
- compte administrateur : compte *authentifié* et *non-vérifié*

Description de l'étape	Captures d'écran
Connexion <u>Espace visiteur authentifié</u> <u>Permet</u> : - accès à la <i>boutique</i> - accès au <i>panier</i>	

Description de l'étape

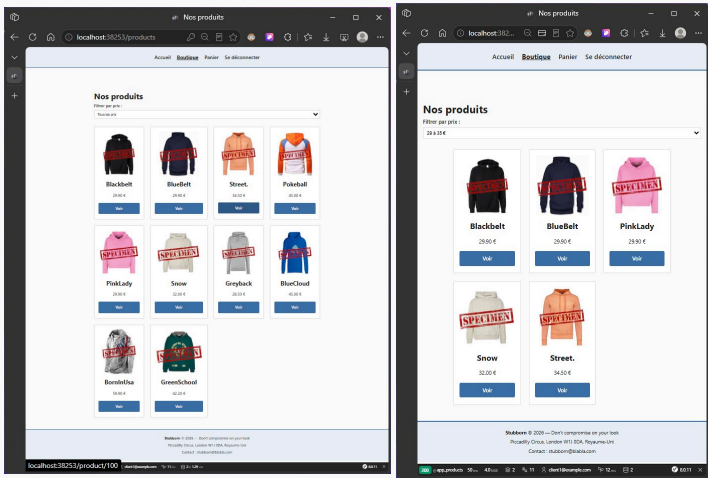
Captures d'écran

Boutique

Espace visiteur authentifié

Permet :

- visualisation des produits
- *filtrage par prix*

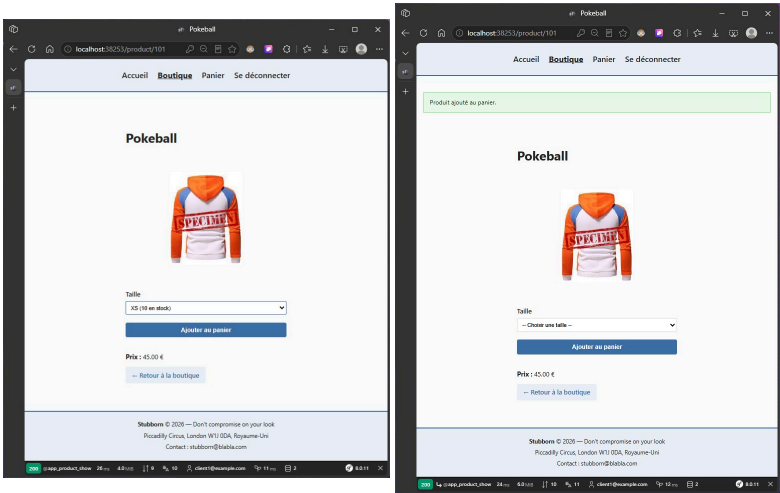


Produit

Espace visiteur authentifié

Permet :

- *sélection de la taille*
- *ajout au panier*



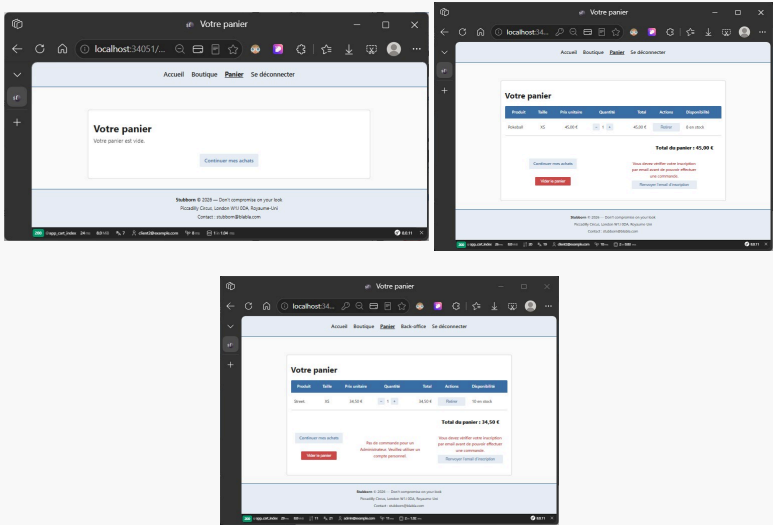
Panier

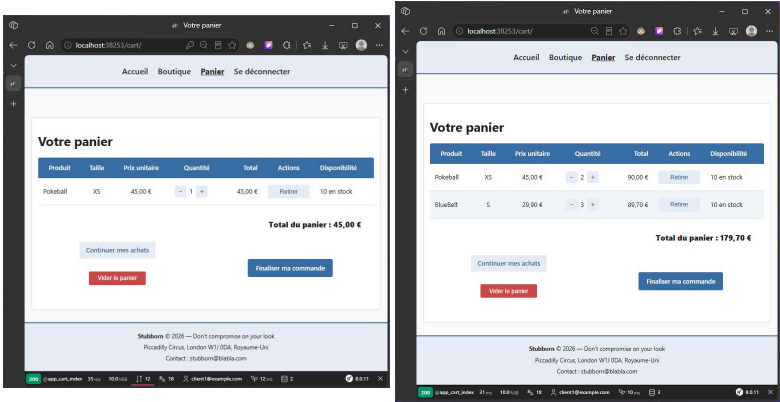
Espace visiteur authentifié

Permet :

- la *visualisation du panier*
- la *gestion du panier*
- d'informer des *conditions d'accès*

Ne permet pas l'achat



Description de l'étape	Captures d'écran
Panier <u>Espace visiteur authentifié inscrit</u> <u>Permet</u> : - la <i>gestion du panier</i> - la <i>finalisation de la commande</i> (Achat)	

9.3 Gestion du panier

Le panier constitue une expérience utilisateur centrale pour un *visiteur authentifié inscrit* :

- l'affichage du panier,
- l'ajout d'un article,
- la diminution d'une quantité,
- la suppression d'un produit,
- le vidage complet du panier,
- la mise à jour automatique du total.

Note

Parcours et expérience utilisateur : Panier → Ajout → Diminution → Suppression → Vidage

Note

Les captures d'écran sont dans le **parcours client** à la section précédente.

9.4 Processus d'achat

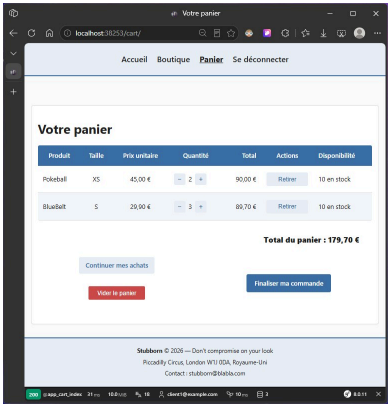
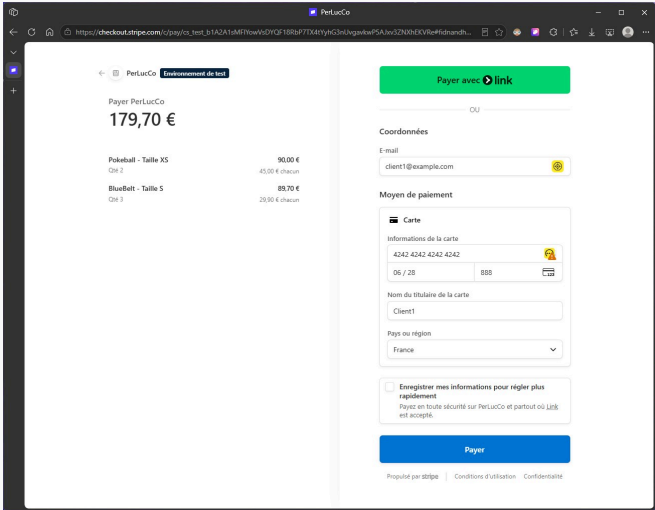
Cette sous-section présente le parcours d'achat complet, depuis le checkout jusqu'au retour Stripe :

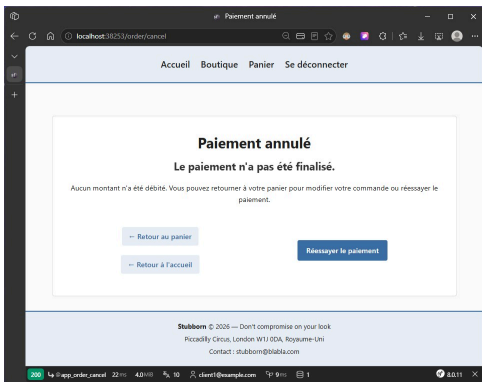
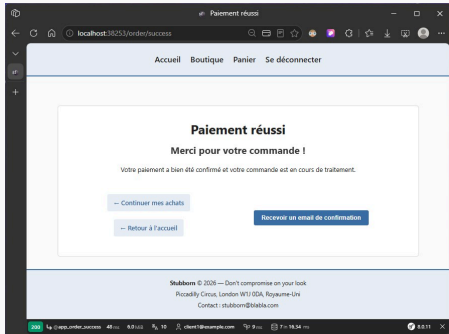
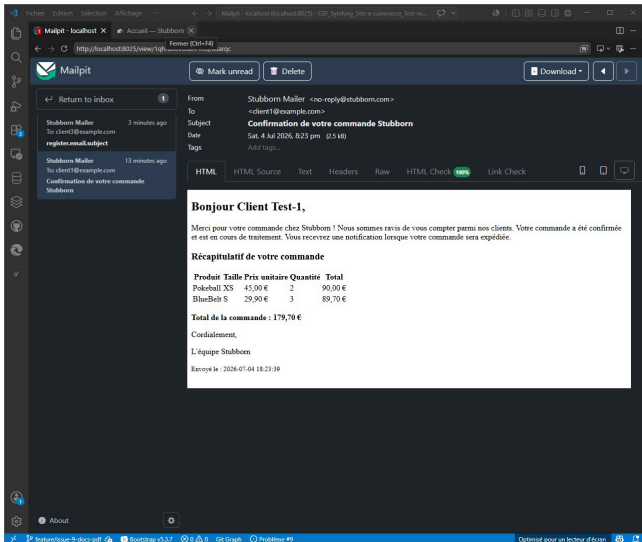
- **Accès au checkout** (utilisateur vérifié uniquement)

- **Redirection vers Stripe Checkout**
- **Paiement simulé en mode test**
- **Retour “Succès”** avec vidage du panier
- **Retour “Annulation”** avec conservation du panier
- **Email de confirmation** via Mailpit

Note

Parcours et expérience utilisateur : Checkout → Stripe → Succès → Annulation → Email de confirmation

Description de l'étape	Capture d'écran
Checkout Espace visiteur authentifié inscrit Permet : - de finaliser la commande - de générer la session Stripe Checkout	
Stripe Espace visiteur authentifié inscrit Permet : - de visualiser la commande à payer - d'annuler le paiement - de compléter le formulaire de paiement - de commander (paiement)	

Description de l'étape	Capture d'écran																				
Annulation <u>Espace visiteur authentifié inscrit</u> <u>Permet</u> : - de <i>visualiser l'annulation</i> de la commande - de <i>poursuivre (réessayer) la commande</i> - de <i>gérer son besoin</i> (panier, accueil)																					
Succès <u>Espace visiteur authentifié inscrit</u> <u>Permet</u> : - de <i>visualiser la réussite</i> de la commande - de <i>recevoir un email</i> de la commande - de <i>poursuivre les achats</i> (boutique) - de <i>gérer son besoin</i> (accueil)																					
Email de confirmation <u>Espace visiteur authentifié inscrit</u> <u>Permet</u> : - dans <i>visualiser la commande</i> dans un email	 <table><tr><th>Produit</th><th>Taille</th><th>Prix unitaire</th><th>Quantité</th><th>Total</th></tr><tr><td>Pokeball XS</td><td></td><td>45,00 €</td><td>2</td><td>90,00 €</td></tr><tr><td>BlueBulb S</td><td></td><td>29,90 €</td><td>3</td><td>89,70 €</td></tr><tr><td colspan="4">Total de la commande :</td><td>179,70 €</td></tr></table>	Produit	Taille	Prix unitaire	Quantité	Total	Pokeball XS		45,00 €	2	90,00 €	BlueBulb S		29,90 €	3	89,70 €	Total de la commande :				179,70 €
Produit	Taille	Prix unitaire	Quantité	Total																	
Pokeball XS		45,00 €	2	90,00 €																	
BlueBulb S		29,90 €	3	89,70 €																	
Total de la commande :				179,70 €																	

9.5 Administration du site

Le back-office réservé à l'administrateur permet :

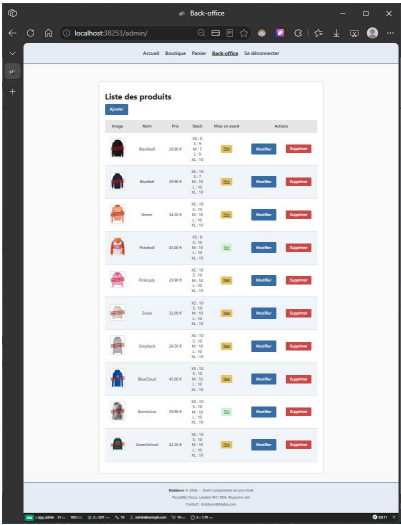
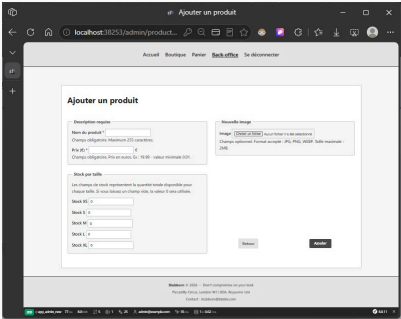
- la consultation du tableau de bord,
- l'ajout d'un produit,
- la modification d'un produit,

- la mise en avant (ou retrait) d'un produit de la page d'accueil,
- la suppression d'un produit.

Les captures d'écran montrent l'interface d'administration et les formulaires CRUD.

Note

Parcours et expérience utilisateur : Back-office → Ajouter un produit → Modifier un produit → Mettre en avant un produit → Supprimer un produit

Description de l'étape	Capture d'écran
Back-office Espace Admin authentifié Permet : - l'ajout d'un produit - la modification d'un produit - la suppression d'un produit - la gestion des mises en avant des produits	
Ajouter un produit Espace Admin authentifié Permet : - la gestion de la description du produit - la gestion de l'image du produit - la gestion du stock par taille Ne permet pas : - la mise en avant du produit	

Description de l'étape

Capture d'écran

Modifier un Produit

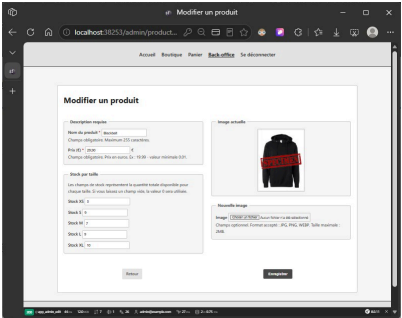
Espace Admin authentifié

Permet :

- la *modification de la description* du produit
- la *modification de l'image* du produit
- la *modification du stock* par taille

Ne permet pas :

- la *gestion de la mise en avant* du produit

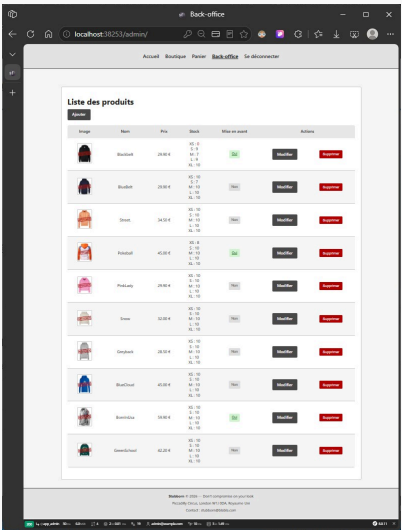
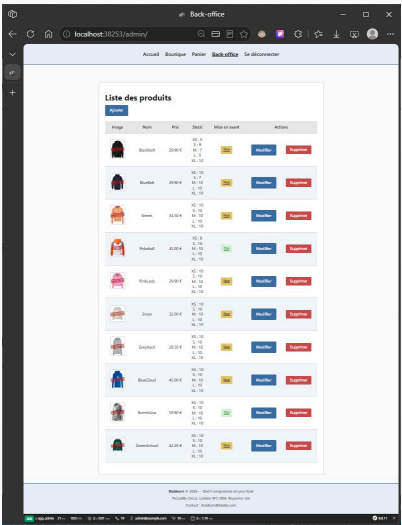


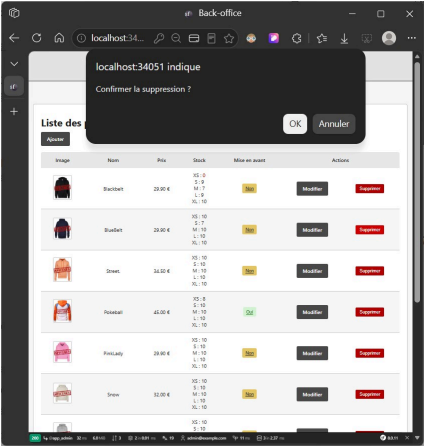
Mettre en avant un Produit

Espace Admin authentifié

Permet :

- la *gestion de la mise en avant* sur la page d'accueil
- la *gestion du nombre maximal* de mise en avant



Description de l'étape	Capture d'écran
Supprimer un Produit <u>Espace Admin authentifié</u> <u>Permet :</u> - d'annuler la suppression du produit - de confirmer réaliser la suppression du produit	

9.6 Lancement de l'application et des tests

Cette sous-section illustre le fonctionnement opérationnel du projet :

9.6.1 Lancement de l'application

- Démarrage des services via `services:start`
- Initialisation de MySQL
- Lancement de Mailpit
- Démarrage du serveur Symfony
- Vérification de l'état des services

Note

Parcours et expérience Développeur : Terminal → `services:start` → Mailpit → Symfony server

Description de l'étape	Capture d'écran
Arrêt de l'application Espace développeur Permet : - arrêt des services - vidage du cache en mémoire	
Démarrage de l'application Espace développeur Permet : - démarrage des services - démarrage de l'application Symfony - chargement des données en base MySQL	

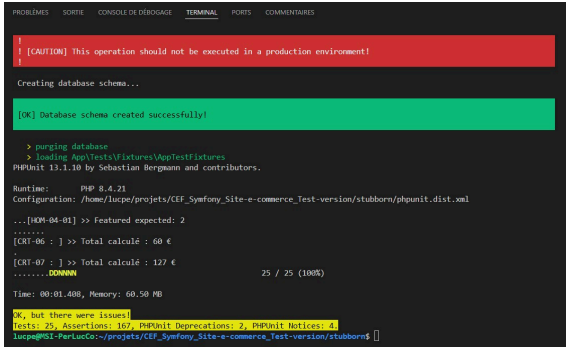
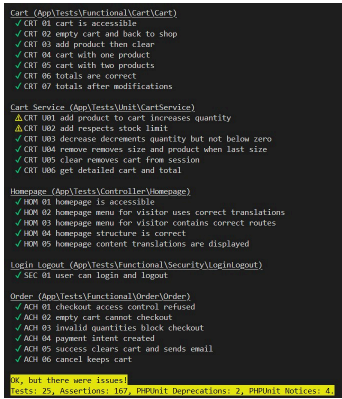
9.6.2 Exécution des tests

- Initialisation de la base de test (`test:init-db`)
- Chargement des fixtures (`test:fixtures`)
- Exécution de la suite complète (`test:run` ou `test:run-verbose`)

- Affichage des résultats PHPUnit

Note

Parcours et expérience Développeur : Terminal → start:test → Résultats PHPUnit

Description de l'étape	Capture d'écran
Tests automatisés <u>Espace développeur</u> <u>Permet</u> : - démarrage de PHPUnit - chargement des données de tests - évaluation des tests dans le terminal <u>Note</u> : - mode <i>normal</i> : <code>composer start:test</code> - mode <i>verbeux</i> : <code>composer start:test-verbose</code>	 <pre>PROBLEMES SORTIE CONSOLE DE DEBOGAGE TERMINAL PORTS COMMENTAIRES [CAUTION] This operation should not be executed in a production environment! Creating database schema... [OK] Database schema created successfully! > purging database > loading App\Tests\Fixtures\AppTestFixtures PHPUnit 13.1.10 by Sebastian Bergmann and contributors. Runtime: PHP 8.4.21 Configuration: /home/lucpe/projects/CEP_Symfony_Site-e-commerce_Test-version/stubborn/phpunit.dist.xml ...[HM-04-01] >> Featured expected: 2 [CR-06 :] >> Total calculé : 60 € [CR-07 :] >> Total calculé : 127 €DOWNNNN 25 / 25 (100%) Time: 00:01.408, Memory: 60.50 MB OK, but there were issues! Tests: 25, Assertions: 107, PHPUnit Deprecations: 2, PHPUnit Notices: 4 lucpage01-ParLucCo-:/projects/CEP_Symfony_Site-e-commerce_Test-version/stubborn\$</pre>  <pre>Cart (App\Tests\Functional\Cart\Cart) ✓ CR1 01 cart is accessible ✓ CR1 02 empty cart and back to shop ✓ CR1 03 add product then clear ✓ CR1 04 cart with one product ✓ CR1 05 cart with two products ✓ CR1 06 totals are correct ✓ CR1 07 totals after modifications Cart Service (App\Tests\Unit\CartService) ▲ CR1 U01 add product to cart increases quantity ▲ CR1 U02 add respects stock limit ✓ CR1 U03 decrease decrements quantity but not below zero ✓ CR1 U04 remove removes size and product when last size ✓ CR1 U05 clear removes cart from session ✓ CR1 U06 get detailed cart and total Homepage (App\Tests\Controller\Homepage) ✓ HM1 01 homepage is accessible ✓ HM1 02 homepage menu for visitor uses correct translations ✓ HM1 03 homepage menu for visitor contains correct routes ✓ HM1 04 homepage structure is correct ✓ HM1 05 homepage content translations are displayed Login Logout (App\Tests\Functional\Security\LoginLogout) ✓ SEC 01 user can login and logout Order (App\Tests\Functional\Order\Order) ✓ AO1 01 checkout access control refused ✓ AO1 02 empty cart cannot checkout ✓ AO1 03 invalid quantities block checkout ✓ AO1 04 payment intent created ✓ AO1 05 success clears cart and sends email ✓ AO1 06 cancel keeps cart OK, but there were issues! Tests: 25, Assertions: 107, PHPUnit Deprecations: 2, PHPUnit Notices: 4</pre>

10. Conclusion

Le projet **Stubborn** marque une étape importante dans la compréhension et la mise en œuvre d’une architecture web professionnelle. Au-delà de la réalisation technique, il illustre la capacité à structurer un projet complet selon les principes du modèle **MVC (Model-View-Controller)**, à exploiter pleinement la puissance du framework **Symfony**, et à concevoir une base réutilisable pour de futurs développements.

Note

Le projet **Stubborn** n'est pas seulement un exercice technique : c'est une **base de travail réutilisable**, un **modèle d'organisation** et une **preuve de maturité professionnelle** dans la conception d'applications web modernes.

10.1 Structuration du projet MVC

L'approche MVC a permis de construire une application claire, évolutive et maintenable. Chaque couche — **modèle, vue, contrôleur** — joue un rôle distinct :

- le **modèle** gère les entités et la logique métier ;
- le **contrôleur** orchestre les interactions et les flux de données ;
- la **vue** présente les informations à l'utilisateur de manière fluide et cohérente.

Cette séparation stricte des responsabilités a facilité la mise en place des tests, la documentation et la compréhension globale du projet. Elle constitue un socle solide pour tout développement professionnel, où la lisibilité et la modularité priment sur la complexité.

10.2 Apport de Symfony

L'emploi de **Symfony8** s'est révélé déterminant pour atteindre une structure efficace et sécurisée sans sur-ingénierie. Le framework offre :

- une **architecture standardisée**, immédiatement exploitable ;
- une **sécurité intégrée** (rôles, pare-feu, vérification email, voter) ;
- une **gestion fluide des services et dépendances** ;
- une **intégration native des tests** et des outils de validation.

Grâce à Symfony, la mise en œuvre de fonctionnalités complexes — authentification, panier, paiement Stripe, back-office — s'est faite de manière naturelle et cohérente. Le framework agit comme un véritable **cadre de travail professionnel**, où chaque composant s'intègre harmonieusement dans l'ensemble.

10.3 Réemploi de l'architecture

L'architecture développée pour **Stubborn** est conçue pour être **réutilisable** et **adaptable** à d'autres projets. Sa modularité permet :

- de remplacer ou d'étendre les entités sans altérer la structure ;
- de réemployer les services (panier, paiement, sécurité) dans d'autres contextes ;
- de conserver la logique de tests et de documentation pour de nouveaux projets.

Cette approche favorise une **évolution continue** : chaque projet peut s'appuyer sur les fondations de Stubborn pour accélérer son développement tout en maintenant un haut niveau de qualité et de sécurité.

10.4 Perspective personnelle

Ce projet a permis de consolider une méthode de travail rigoureuse : concevoir, structurer, tester, documenter. Il démontre qu'un projet bien organisé, fondé sur une architecture claire et des outils fiables, peut évoluer vers une solution professionnelle complète.

L'expérience acquise avec Symfony ouvre la voie à des projets plus ambitieux, où la maîtrise du framework et la rigueur du modèle MVC deviennent des atouts majeurs.

11. Annexes

Les annexes présentent les éléments techniques complémentaires du projet :

1. organisation du dépôt
2. scripts Composer
3. arborescence du code
4. extraits de codage significatifs.

Elles permettent d'approfondir la compréhension de l'architecture et de la logique applicative sans surcharger les sections principales.

11.1 Organisation du dépôt

11.1.1 Décomposition en milestones

Le développement du projet a été structuré en **milestones GitHub**, chacune correspondant à une phase clé du travail. Cette organisation a permis de suivre la progression, de valider les livrables et de garantir la cohérence du développement.

Milestone	Objectif principal	Contenu
#1 — Organisation	Mise en place du dépôt, configuration Symfony, environnement local.	Création du squelette du projet et des premières entités.
#2 — Développement	Développement des fonctionnalités de l'application	Implémentation de la sécurité, des entités, des routes, des services métier, des données métiers et des templates de page et des styles.
#3 — Tests	Développement des tests	Mise en place des tests automatisés et des scripts <code>composer</code>
#4 — Documentation & Livraison	Finalisation du livrable et validation des tests.	Consolidation de la documentation et préparation du rendu final.

Note

Les milestones sont visibles dans le dépôt GitHub sous l'onglet **Issues** → **Milestones**, et chacune regroupe les tickets correspondants à ses tâches.

11.1.2 Organisation des issues

Les **issues** détaillent les tâches unitaires du projet. Elles suivent une nomenclature précise pour assurer la traçabilité :

- **Organisation**
 - **issue-1** : Plan de réalisation et README
- **Développement**
 - **issue-2** : Setup Symfony - Entités - Fixtures
 - **issue-3** : Authentification : login, register, email d'activation

- **issue-4** : Catalogue produits et fiche produit
- **issue-5** : Panier en session
- **issue-6** : Paiement Stripe (mode test)
- **issue-7** : Back-office administrateur - CSS global
- **Tests**
 - **issue-8** : Tests unitaires et fonctionnels (Panier et Achat)
- **Documentation & Livraison**
 - **issue-9** : Documentation projet (PDF)
 - **issue-10** : Documentation GitHub
 - **issue-11** : Livraison finale

Chaque issue contient:

- une **description synthétique** de la tâche;
- les **étapes de développement**;
- les **commits associés**;
- les **liens vers les branches** correspondantes (`feature/issue-x`).

11.1.3 Kanban de gestion du planning

Le suivi du projet est assuré par un **tableau Kanban GitHub Projects**, accessible à l'adresse :

 <https://github.com/users/MonLucCo/projects/12/views/1>

Ce tableau permet de visualiser l'avancement des tâches selon trois colonnes principales :

Colonne	Description
To Do	Tâches planifiées mais non commencées.
In Progress	Tâches en cours de développement ou de test.
In Review	Tâches en d'évaluation de la branche <code>dev</code> pour intégration dans <code>main</code>
Done	Tâches terminées et validées.

Le Kanban offre une vue synthétique du planning et facilite la gestion du temps et des priorités.

11.1.4 GitGraph du développement

Le **GitGraph** du dépôt illustre la chronologie du développement et la structure des branches :



Chaque branche **feature/issue-X** correspond à une tâche isolée, développée puis fusionnée dans la branche **dev**, avant validation finale sur **main**.

 Note

Cette organisation GitHub — *milestones, issues, Kanban* et *Git Graph* — constitue la **colonne vertébrale du projet Stubborn**. Elle garantit une progression structurée, une visibilité complète sur le développement et une documentation parfaitement alignée avec le code source.

11.2 Scripts de composer

Les scripts définis dans le fichier `composer.json` automatisent les opérations courantes du projet : initialisation, chargement des fixtures, exécution des tests et gestion des services.

Script	Commande exécutée	Objectif
<code>app:fixtures</code>	<code>php bin/console doctrine:fixtures:load --env=dev --group=dev --no-interaction</code>	Charger les données de développement.
<code>test:init-db</code>	<code>php bin/console doctrine:schema:create --env=test</code>	Créer la base de données de test.
<code>test:fixtures</code>	<code>php bin/console doctrine:fixtures:load --env=test --group=test --no-interaction</code>	Injecter les fixtures de test.
<code>test:run</code>	<code>@test:init-db</code> → <code>@test:fixtures</code> → <code>php bin/phpunit</code>	Exécuter la suite complète de tests.
<code>services:start</code>	Démarrage de MySQL, Mailpit, Messenger et serveur Symfony.	Initialiser l'environnement applicatif.
<code>services:stop</code>	Arrêt des services actifs.	Libérer les ressources locales.
<code>services:restart</code>	Stop → Clear cache → Dump autoload → Start.	Réinitialiser proprement l'environnement.
<code>start:test</code>	Redémarrage des services + exécution des tests.	Vérification complète du projet.

Note

Ces scripts ne constituent pas un axe majeur du développement, mais sont essentiels pour **l'exploitation et la reproductibilité** des travaux. Ils garantissent la cohérence entre les environnements `dev` et `test`.

11.3 Arborescence du projet et descriptif

Cette annexe présente **l'arborescence réelle du projet Stubborn**, telle qu'elle existe dans le dépôt GitHub. Chaque dossier et fichier est accompagné d'un descriptif succinct permettant de comprendre son rôle dans l'architecture Symfony.

Arborescence détaillée

stubborn/	--> Racine du projet Symfony
├─ bin/	--> Scripts exécutables (console Symfony)
└─ console	--> Interface CLI pour commandes Symfony
├─ config/	--> Configuration globale du framework
├─ packages/	--> Config des bundles (Doctrine, Twig, Mailer...)
├─ routes/	--> Définition des routes (annotations, YAML)
└─ services.yaml	--> Déclaration des services + autowiring
├─ public/	--> Point d'entrée HTTP du site
├─ index.php	--> Front controller Symfony
├─ css/	--> Styles (admin)
└─ images/	--> Ressources publiques (images, téléchargements)
├─ src/	--> Code applicatif (MVC + services + sécurité)
├─ Controller/	--> Contrôleurs : logique de navigation
├─ AccountController.php	--> Gestion de la vérification de compte
├─ AdminController.php	--> Back-office administrateur (CRUD produits)
├─ CartController.php	--> Gestion du panier
├─ HomeController.php	--> Page d'accueil
├─ OrderController.php	--> Processus d'achat + Stripe
├─ ProductController.php	--> Catalogue + fiche produit
├─ ProductsController.php	--> Boutique des produits
└─ SecurityController.php	--> Inscription, connexion, vérification email
├─ DataFixtures/	--> Chargement des données
└─ AppFixtures.php	--> Chargement des données en base MySQL
├─ Entity/	--> Entités Doctrine (modèle métier)
├─ Product.php	--> Produit : nom, prix, stock, tailles
└─ User.php	--> Utilisateur : email, rôles, vérification
├─ Form/	--> Formulaires
├─ ProductType.php	--> Construction du formulaire du produit
└─ RegistrationFormType.php	--> Construction du formulaire d'inscription
├─ Repository/	--> Requêtes Doctrine personnalisées
└─ ProductRepository.php	--> Accès aux produits (tri, filtrage)
├─ Security/	--> Composants de sécurité
└─ voter/	

	└─ IsVerifiedVoter.php	--> Vérifie que l'utilisateur est activé
	└─ service/	--> Services métiers réutilisables
	└─ CartService.php	--> Gestion du panier en session
	└─ OrderService.php	--> Gestion de la commande en session Stripe
	└─ ProductService.php	--> Gestion des produits (CRUD)
	└─ StripeService.php	--> Création de la session Stripe Checkout
	└─ kernel.php	--> Noyau principal de l'application Symfony
	└─ templates/	--> Vues Twig (interface utilisateur)
	└─ base_admin.html.twig	--> Template principal du back-office
	└─ base.html.twig	--> Template principal du site
	└─ account/	--> Page utilisateur non vérifié
	└─ not_verified.html.twig	--> Template utilisateur non vérifié
	└─ cart/	--> Pages panier
	└─ index.html.twig	--> Contenu du panier
	└─ email/	--> Template de l'email d'achat
	└─ order_confirmation.html.twig	--> Version HTML de l'email
	└─ order_confirmation.txt.twig	--> Version Texte de l'email
	└─ form/	--> Template du formulaire
	└─ _theme.html.twig	--> Visualisation du formulaire
	└─ home/	--> Pages accueil
	└─ index.html.twig	--> Contenu de la page d'accueil
	└─ order/	--> Pages achat + retours Stripe
	└─ success.html.twig	--> Paiement réussi
	└─ cancel.html.twig	--> Paiement annulé
	└─ partials/	--> Eléments partiels de page
	└─ _footer.html.twig	--> Pied de page
	└─ _header.html.twig	--> Entête de page et menu de navigation
	└─ product/	--> Pages catalogue + fiche produit
	└─ show.html.twig	--> Fiche produit
	└─ products/	--> Pages boutique
	└─ index.html.twig	--> Liste des produits
	└─ registration/	--> Page inscription
	└─ confirmation_email.html.twig	--> Création de l'email d'inscription
	└─ register.html.twig	--> Inscription
	└─ security/	--> Page connexion
	└─ login.html.twig	--> Connexion
	└─ admin/	--> Back-office
	└─ index.html.twig	--> Tableau de bord admin
	└─ product_form.html.twig	--> Formulaire CRUD produit
	└─ tests/	--> Tests unitaires et fonctionnels
	└─ webTestCaseBase.php	--> Classe de base commune aux tests
	└─ Controller/	--> Tests pages publiques
	└─ HomepageTest.php	--> Tests HOM-01 à HOM-03
	└─ Fixtures/	--> Données de tests
	└─ AppTestFixtures.php	--> Chargement des données de tests
	└─ Functional/	--> Tests des fonctions (panier, achat, sécurité)
	└─ Cart/	--> Fonction panier
	└─ CartTest.php	--> Tests CRT-01 à CRT-05
	└─ Order/	--> Fonction achat

└─ OrderTest.php	--> Tests ACH-01 à ACH-06
└─ Security/	--> Fonction accès sécurisé (login/logout)
└─ OrderTest.php	--> Tests SEC-01
└─ var/	--> Cache, logs, base SQLite test
└─ cache/	--> Cache Symfony
└─ log/	--> Logs applicatifs
└─ tests/	--> Base de données de test (SQLite, uploads)
└─ vendor/	--> Dépendances Composer (Symfony, Stripe...)
└─ docs/	--> Documentation du projet
└─ documentation-projet.pdf	--> Document du projet (livrable)
└─ sources/	--> Sources de la documentation
└─ documentation-projet.md	--> Document du projet (source Typora)
└─ annexes/	--> Annexes techniques
└─ ressources/	--> Images, schémas, captures d'écran
└─ composer.json	--> Dépendances + scripts d'exploitation
└─ composer.lock	--> Version figée des dépendances
└─ .env	--> Variables d'environnement (dev)
└─ .env.local	--> Variables d'environnement (langue, Stripe,
MySQL)	
└─ .env.test	--> Variables d'environnement (test)

Note

Cette structure respecte le modèle MVC et facilite la maintenance du projet. Les tests sont isolés dans les dossiers `tests/` (code de tests) et `var/tests/` (données de tests), garantissant une séparation nette entre le code applicatif et la validation fonctionnelle.

Note

Les variables d'environnement personnelles `.env.local` définissent le langage d'affichage (Français ou Anglais), les données de connexion à la base **MySQL** et au compte **Stripe Checkout** (en mode test).

Note

La documentation a été réalisée en premier lieu en *Mark Down* avec l'éditeur **Typora**, puis exportée en **PDF** pour établir la version livrable.

Les annexes techniques constituent des documents établis lors du développement et qui ont permis l'élaboration du document final (livrable).

11.4 Extraits de codes importants

Les extraits suivants illustrent les principes de codage évoqués dans les sections 6 et 7. Ils montrent la mise en œuvre concrète des services, contrôleurs et tests.

11.4.1 Service CartService

```
class CartService
{
    private ?SessionInterface $session;
    private ProductRepository $productRepository;

    public function __construct(RequestStack $requestStack, ProductRepository $productRepository)
    {
        $this->session = $requestStack->getSession();
        $this->productRepository = $productRepository;
    }

    private function getCart(): array
    {
        return $this->session->get('cart', []);
    }

    private function saveCart(array $cart): void
    {
        $this->session->set('cart', $cart);
    }

    public function add(int $productId, string $size): string
    {
        // validation de la taille
        if (!in_array($size, Product::SIZES, true)) {
            return 'invalid_size';
        }

        $cart = $this->getCart();
        $product = $this->productRepository->find($productId);

        if (!$product) {
            return 'product_not_found';
        }

        // vérification du stock pour cette taille
        $maxStock = $product->getStockForSize($size);
        if ($maxStock === null) {
            return 'size_not_available';
        }
    }
}
```

```

// Si le produit n'existe pas encore dans le panier
if (!isset($cart[$productId])) {
    $cart[$productId] = [];
}

// Si la taille n'existe pas encore
if (!isset($cart[$productId][$size])) {
    $cart[$productId][$size] = ['quantity' => 0];
}

// Quantité actuelle
$currentQty = $cart[$productId][$size]['quantity'];

// Vérification du stock
if ($currentQty < $maxStock) {
    $cart[$productId][$size]['quantity']++;
} else return 'stock_limit_reached';

$this->saveCart($cart);

return 'added';
}

public function decrease(int $id, string $size): array
{
    $cart = $this->getCart();

    if (!isset($cart[$id])) {
        return ['status' => 'product_not_in_cart'];
    }

    if (!isset($cart[$id][$size])) {
        return ['status' => 'size_not_in_cart'];
    }

    $currentQty = $cart[$id][$size]['quantity'];

    if ($currentQty === 0) {
        return [
            'status' => 'quantity_already_zero',
            'quantity' => 0
        ];
    }

    // Diminuer la quantité
    $cart[$id][$size]['quantity']--;

    $this->saveCart($cart);

    return [
        'status' => 'decreased',
        'quantity' => $cart[$id][$size]['quantity']
    ];
}

```

```

}

public function remove(int $productId, string $size): string
{
    $cart = $this->getCart();

    // Si l'entrée n'existe pas, on ne touche à rien
    if (!isset($cart[$productId])) {
        return 'product_not_in_cart';
    }

    if (!isset($cart[$productId][$size])) {
        return 'size_not_in_cart';
    }

    unset($cart[$productId][$size]);

    // Si plus aucune taille pour ce produit → supprimer le produit
    if (empty($cart[$productId])) {
        unset($cart[$productId]);
    }

    $this->saveCart($cart);

    return 'removed';
}

public function clear(): string
{
    $this->session->remove('cart');

    return 'cleared';
}

public function getDetailedCart(): array
{
    $cart = $this->getCart();
    $detailedCart = [];

    foreach ($cart as $productId => $sizes) {
        $product = $this->productRepository->find($productId);

        if (!$product) {
            continue;
        }

        foreach ($sizes as $size => $data) {
            $quantity = $data['quantity'];

            $detailedCart[] = [
                'product' => $product,
                'size' => $size,
                'price' => $product->getPrice(),
            ];
        }
    }

    return $detailedCart;
}

```

```

                'quantity' => $quantity,
                'total' => $product->getPrice() * $quantity,
            ];
        }
    }

    return $detailedCart;
}

public function getTotal(array $items): float
{
    $total = 0;

    foreach ($items as $item) {
        $total += $item['total'];
    }

    return $total;
}

public function saveLastOrder(array $items, float $total): void
{
    $this->session->set('last_order_items', $items);
    $this->session->set('last_order_total', $total);
}

public function getLastOrderItems(): array
{
    return $this->session->get('last_order_items', []);
}

public function getLastOrderTotal(): float
{
    return $this->session->get('last_order_total', 0);
}

public function clearLastOrder(): void
{
    $this->session->remove('last_order_items');
    $this->session->remove('last_order_total');
}

public function getProduct(int $id): ?Product
{
    return $this->productRepository->find($id);
}
}

```

11.4.2 Service StripeService

```
class StripeService
{
    private UrlGeneratorInterface $urlGenerator;
    private CartService $cartService;

    private TranslatorInterface $translator;

    public function __construct(
        UrlGeneratorInterface $urlGenerator,
        CartService $cartService,
        TranslatorInterface $translator
    ) {
        $this->urlGenerator = $urlGenerator;
        $this->cartService = $cartService;
        $this->translator = $translator;

        // Charger la clé secrète Stripe
        Stripe::setApiKey($_ENV['STRIPE_SECRET_KEY']);
    }

    /**
     * Crée une session Stripe Checkout
     */
    public function createCheckoutSession(array $lineCartItems): Session
    {
        // Stripe pour les tests (utilisation d'une session Stripe sans API)
        if ($_ENV['APP_ENV'] === 'test') {
            return Session::constructFrom([
                'id' => 'cs_test_fake',
                'url' => 'https://checkout.stripe.com/test-session',
            ]);
        }

        // Stripe pour de développement et la production (Checkout Stripe)
        $lineStripeItems = $this->getStripeLineItems($lineCartItems);

        if (empty($lineStripeItems)) {
            throw new \InvalidArgumentException($this->translator-
>trans('cart.flash.empty_quantities'));
        }

        // Docs :Stripe\Checkout\Session::create() -
        //      https://stripe.com/docs/api/checkout/sessions/create#create_checkout_session-
        locale
        $allowedLocales = ['auto', 'fr', 'en', 'de', 'it', 'nl', 'pt', 'sv', 'da', 'fi', 'no', 'ja',
        'zh'];
        $locale = $_ENV['APP_DEFAULT_LOCALE'] ?? 'auto';

        if (!in_array($locale, $allowedLocales, true)) {
            $locale = 'auto';
        }
    }
}
```

```

    }

    return Session::create([
        'payment_method_types' => ['card'],
        'mode' => 'payment',
        'line_items' => $lineStripeItems,
        'success_url' => $this->urlGenerator->generate(
            'app_order_success', [],
            UrlGeneratorInterface::ABSOLUTE_URL),
        'cancel_url' => $this->urlGenerator->generate(
            'app_order_cancel', [],
            UrlGeneratorInterface::ABSOLUTE_URL),

        'locale' => $locale,
    ]);
}

/**
 * Récupère les éléments de ligne Stripe
 */
public function getStripeLineItems(array $lineCartItems): array
{
    $items = [];

    foreach ($lineCartItems as $cartItem) {
        $product = $cartItem['product'];
        $quantity = $cartItem['quantity'];
        $size = $cartItem['size'];

        if ($quantity < 1) {
            continue; // Ignorer les quantités nulles ou négatives
        }

        $translatedSize = sprintf(
            $this->translator->trans(
                'order.stripe.size',
                ['%size%' => $size]
            )
        );
        $productName = $product->getName() . ' - ' . $translatedSize;

        $items[] = [
            'price_data' => [
                'currency' => 'eur',
                'product_data' => [
                    'name' => $productName,
                ],
                'unit_amount' => $product->getPrice() * 100, // Stripe = centimes
            ],
            'quantity' => $quantity,
        ];
    }

    return $items;
}

```

```
}  
}
```

11.4.3 Contrôleur OrderController

```
class OrderController extends AbstractController  
{  
  #[Route('/order/checkout', name: 'app_order_checkout')]  
  public function checkout(  
    OrderService $ordersService  
  ): Response {  
  
    // Contrôle des accès direct  
    // Empêcher les admins de passer commande  
    if ($this->isGranted('ROLE_ADMIN')) {  
      $this->addFlash('error', 'admin.flash.order_forbidden');  
      return $this->redirectToRoute('app_admin');  
    }  
    // Vérifier que l'utilisateur est connecté et vérifié  
    if (!$this->getUser()) {  
      $this->addFlash('error', 'order.flash.login_required');  
      return $this->redirectToRoute('app_home');  
    }  
    if (!$this->isGranted('IS_VERIFIED')) {  
      $this->addFlash('error', 'order.flash.user_not_verified');  
      return $this->redirectToRoute('app_account_not_verified');  
    }  
  
    $user = $this->getUser();  
  
    try {  
      $items = $ordersService->prepareCheckout($user);  
    } catch (\RuntimeException $e) {  
      if ($e->getMessage() === 'empty_cart') {  
        $this->addFlash('warning', 'order.flash.empty_quantities');  
        return $this->redirectToRoute('app_cart_index');  
      }  
      throw $e; // Re-throw unexpected exceptions  
    }  
  
    // 2) Appel Stripe  
    $session = $ordersService->createStripeSession($items);  
    return $this->redirect($session->url);  
  }  
  
  #[Route('/order/success', name: 'app_order_success')]  
  public function success(  
    OrderService $ordersService  
  ): Response {  
    // Contrôle des accès direct  
    // Vérifier que l'utilisateur est connecté et vérifié
```

```

        if (!$this->getUser()) {
            return $this->redirectToRoute('app_home');
        }
        if (!$this->isGranted('IS_VERIFIED')) {
            return $this->redirectToRoute('app_account_not_verified');
        }

        $user = $this->getUser();
        $orderService->processSuccess($user);

        return $this->render('order/success.html.twig');
    }

#[Route('/order/cancel', name: 'app_order_cancel')]
public function cancel(): Response
{
    // Contrôle des accès direct
    // Vérifier que l'utilisateur est connecté et vérifié
    if (!$this->getUser()) {
        return $this->redirectToRoute('app_home');
    }
    if (!$this->isGranted('IS_VERIFIED')) {
        return $this->redirectToRoute('app_account_not_verified');
    }

    return $this->render('order/cancel.html.twig');
}

#[Route('/order/send-confirmation', name: 'app_order_send_confirmation', methods: ['POST'])]
public function sendConfirmation(
    OrderService $orderService
): Response {
    if (!$this->getUser()) {
        return $this->redirectToRoute('app_home');
    }

    $user = $this->getUser();
    $orderService->sendConfirmationEmail($user);

    $this->addFlash('success', 'order.flash.success_order');

    return $this->redirectToRoute('app_products');
}
}

```


11.4.4 Test ACH-04 — Création de session Stripe simulée

```
/* =====
Helpers spécifiques aux tests d'achat
===== */

private function goToCheckout(): void
{
    $crawler = $this->client->request('GET', '/cart' . '/');
    $link = $crawler->filter('[data-test="cart-btn-checkout"]')->link();
    $this->crawler = $this->client->click($link);
}

...

/* =====
ACH-04 : Création PaymentIntent Stripe
===== */
public function test_ACH_04_payment_intent_created()
{
    $this->loginAsUser();
    $this->addProduct(1, 'M');

    // Aller au checkout via l'UI
    $this->goToCheckout();

    // Vérifier qu'on a bien une redirection
    $this->assertResponseRedirects();

    // Vérifier que l'URL de redirection pointe vers stripe
    $location = $this->client->getResponse()->headers->get('Location');
    $this->assertNotNull($location);
    $this->assertStringStartsWith('https://checkout.stripe.com', $location);
}
```

11.4.5 Voter IS_VERIFIED

```
class IsVerifiedVoter extends Voter
{
    protected function supports(string $attribute, mixed $subject): bool
    {
        return $attribute === 'IS_VERIFIED';
    }

    protected function voteOnAttribute(
        string $attribute,
        mixed $subject,
        TokenInterface $token,
        ?Vote $vote = null
    ) {
    }
```

```
    ): bool {  
        $user = $token->getUser();  
  
        if (!$user instanceof User) {  
            return false;  
        }  
  
        return $user->isverified();  
    }  
}
```

Note

Ces extraits illustrent les **principes de codage** appliqués dans le projet : modularité, clarté, séparation des responsabilités et validation systématique. Ils complètent les explications des **sections 6 et 7**, en offrant une vision concrète du développement.